When $c > v$ (fighting is costly relative to the resource), the game has no pure-strategy Nash equilibrium. The mixed-strategy Nash equilibrium has each player playing Hawk with probability:

$$p^* = \frac{v}{c} \tag{35.1}$$

This is also the evolutionarily stable strategy (ESS): a population at frequency p^* cannot be invaded by either pure strategy (Osborne, 2004).

35.2.2. Agent-based evolutionary dynamics

In the ABM framework, we model a finite population of N agents. Each generation proceeds in three steps:

1. **Interaction.** Each agent plays the Hawk-Dove game against k randomly chosen opponents. Its fitness is the average payoff across these interactions.

2. **Reproduction.** The next generation is formed by sampling N agents from the current population with replacement, where the probability of being selected is proportional to fitness (shifted to be non-negative):

$$\Pr(\text{agent } i \text{ selected}) = \frac{f_i - f_{\min} + \epsilon}{\sum_{j=1}^{N}(f_j - f_{\min} + \epsilon)} \tag{35.2}$$

where f_i is agent i's fitness, $f_{\min}$ is the minimum fitness in the population, and $\epsilon > 0$ is a small constant ensuring all agents have positive selection probability.

3. **Mutation.** Each offspring independently switches strategy with probability μ (the mutation rate).

> **Definition**
>
> An **agent-based model** (ABM) is a computational model in which autonomous agents with individual attributes interact according to specified rules. Macro-level patterns emerge from these micro-level interactions without being explicitly programmed.

35.2.3. Connection to replicator dynamics

In the limit of large populations, weak selection, and low mutation, the ABM dynamics converge to the **replicator equation** (Chapter 41):

$$\dot{x}_H = x_H \left[\pi_H(\mathbf{x}) - \bar{\pi}(\mathbf{x})\right] \tag{35.3}$$

where x_H is the frequency of Hawks, π_H is the expected payoff to a Hawk, and $\bar{\pi}$ is the population average payoff. The ABM allows us to see how finite population effects, stochastic drift, and mutation perturb these deterministic predictions.

35.3. Implementation in R

35.3.1. Core simulation functions

```r
# Hawk-Dove payoff function
hawk_dove_payoff <- function(strategy1, strategy2, v = 2, c = 5) {
  if (strategy1 == "Hawk" && strategy2 == "Hawk") {
    return((v - c) / 2)
  } else if (strategy1 == "Hawk" && strategy2 == "Dove") {
    return(v)
  } else if (strategy1 == "Dove" && strategy2 == "Hawk") {
    return(0)
  } else {
    return(v / 2)
  }
}

# Compute fitness for all agents via random pairwise interactions
compute_fitness <- function(population, n_interactions, v, c) {
  n <- length(population)
  fitness <- numeric(n)
  for (i in seq_len(n)) {
    opponents <- sample(seq_len(n)[-i], min(n_interactions, n - 1))
    payoffs <- vapply(opponents, function(j) {
      hawk_dove_payoff(population[i], population[j], v, c)
    }, numeric(1))
    fitness[i] <- mean(payoffs)
  }
  fitness
}

# Fitness-proportional selection with shift
select_parents <- function(population, fitness, n) {
  shifted <- fitness - min(fitness) + 0.01
  probs <- shifted / sum(shifted)
  idx <- sample(seq_along(population), n, replace = TRUE, prob = probs)
  population[idx]
}

# Mutation
mutate_population <- function(population, mu) {
  strategies <- c("Hawk", "Dove")
  mutate_mask <- runif(length(population)) < mu
  population[mutate_mask] <- vapply(population[mutate_mask], function(s) {
    sample(setdiff(strategies, s), 1)
  }, character(1))
  population
}
```

35.3.2. Running the ABM

```
run_hawk_dove_abm <- function(n_agents = 200, n_generations = 300,
                              initial_hawk_freq = 0.5,
                              n_interactions = 10,
                              mutation_rate = 0.01,
                              v = 2, c = 5, seed = 42) {
  set.seed(seed)

  # Initialize population
  n_hawks <- round(n_agents * initial_hawk_freq)
  population <- c(rep("Hawk", n_hawks), rep("Dove", n_agents - n_hawks))

  # Storage for tracking
  history <- tibble(
    generation = integer(),
    hawk_freq = double(),
    mean_fitness = double()
  )

  for (gen in seq_len(n_generations)) {
    hawk_freq <- mean(population == "Hawk")
    fitness <- compute_fitness(population, n_interactions, v, c)

    history <- bind_rows(history, tibble(
      generation = gen,
      hawk_freq = hawk_freq,
      mean_fitness = mean(fitness)
    ))

    # Selection and reproduction
    population <- select_parents(population, fitness, n_agents)

    # Mutation
    population <- mutate_population(population, mutation_rate)
  }

  history
}
```

35.3.3. Strategy frequency evolution

```
v <- 2
c_val <- 5
ess_freq <- v / c_val

# Run three independent simulations with different initial conditions
runs <- list(
```

```
  list(init = 0.1, seed = 42, label = "Run 1 (10% Hawk)"),
  list(init = 0.5, seed = 123, label = "Run 2 (50% Hawk)"),
  list(init = 0.9, seed = 456, label = "Run 3 (90% Hawk)")
)

history_all <- map_dfr(runs, function(r) {
  run_hawk_dove_abm(
    n_agents = 200, n_generations = 300,
    initial_hawk_freq = r$init, n_interactions = 10,
    mutation_rate = 0.01, v = v, c = c_val, seed = r$seed
  ) |>
    mutate(run = r$label)
})

p_evolution <- ggplot(history_all, aes(x = generation, y = hawk_freq,
                                        colour = run)) +
  geom_line(linewidth = 0.6, alpha = 0.85) +
  geom_hline(yintercept = ess_freq, linetype = "dashed",
             colour = okabe_ito[8], linewidth = 0.6) +
  scale_colour_manual(values = okabe_ito[c(1, 2, 3)], name = "Simulation") +
  theme_publication() +
  labs(title = "Hawk-Dove ABM: Strategy Frequency Over Generations",
       x = "Generation",
       y = "Fraction playing Hawk",
       caption = glue("Dashed line = ESS frequency (v/c = {ess_freq})"))

p_evolution
save_pub_fig(p_evolution, "abm-strategy-evolution", width = 7, height = 5)
```

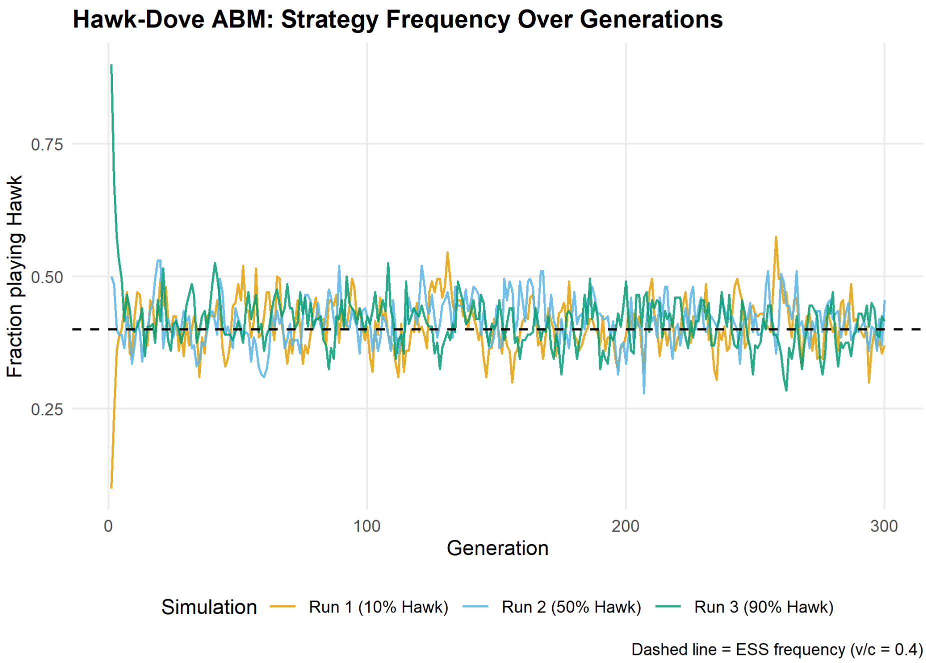

Figure 35.1.: Strategy frequency evolution in the Hawk-Dove agent-based model across three independent runs. The dashed horizontal line marks the analytical ESS frequency $p^* = v/c = 0.4$. Despite starting from different initial conditions, all runs converge to fluctuate around the predicted equilibrium.

35.3.4. Population fitness over time

```
# Compute the analytical equilibrium fitness
# At ESS: pi_bar = p*[p*(v-c)/2 + (1-p*)v] + (1-p*)[p*0 + (1-p*)v/2]
p_star <- ess_freq
pi_bar_star <- p_star * (p_star * (v - c_val)/2 + (1 - p_star) * v) +
               (1 - p_star) * (p_star * 0 + (1 - p_star) * v/2)

p_fitness <- ggplot(history_all, aes(x = generation, y = mean_fitness,
                                      colour = run)) +
  geom_line(linewidth = 0.6, alpha = 0.85) +
  geom_hline(yintercept = pi_bar_star, linetype = "dashed",
             colour = okabe_ito[8], linewidth = 0.6) +
  scale_colour_manual(values = okabe_ito[c(1, 2, 3)], name = "Simulation") +
  theme_publication() +
  labs(title = "Hawk-Dove ABM: Average Population Fitness",
       x = "Generation",
       y = "Mean payoff per interaction",
       caption = glue("Dashed line = fitness at ESS ({round(pi_bar_star, 2)})"))

p_fitness
save_pub_fig(p_fitness, "abm-fitness", width = 7, height = 5)
```

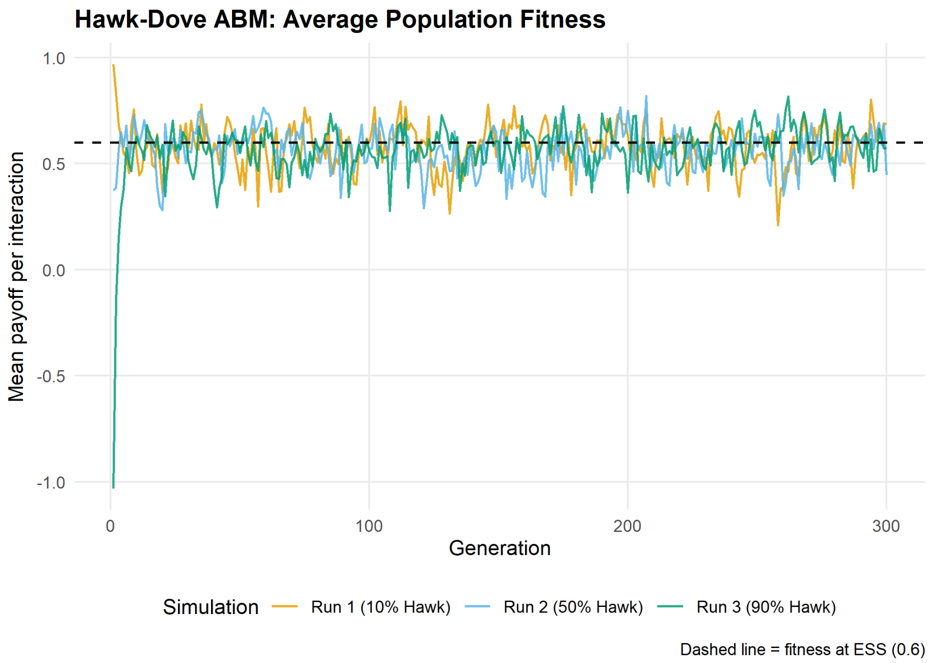

Figure 35.2.: Average population fitness over generations in the Hawk-Dove ABM. Fitness is highest when the population is near the ESS. Early generations show rapid fitness changes as the population adjusts from its initial state; later generations fluctuate around the equilibrium fitness level.

35.4. Worked example

Let us trace through the first few generations of a small population to understand the mechanics.

```
set.seed(42)
n_agents <- 20
population <- c(rep("Hawk", 14), rep("Dove", 6))  # 70% Hawk

cat("Initial population (20 agents, 70% Hawk):\n")
```

```
#> Initial population (20 agents, 70% Hawk):
```

```
cat(glue("  Hawks: {sum(population == 'Hawk')}, ",
         "Doves: {sum(population == 'Dove')}"), "\n\n")
```

```
#>   Hawks: 14, Doves: 6
```

```
# Generation 1: compute fitness
fitness <- compute_fitness(population, n_interactions = 5, v = v, c = c_val)

fitness_by_type <- tibble(
  strategy = population,
  fitness = fitness
```

```
) |>
  group_by(strategy) |>
  summarise(mean_fitness = round(mean(fitness), 3),
            min_fitness = round(min(fitness), 3),
            max_fitness = round(max(fitness), 3),
            .groups = "drop")

cat("Generation 1 fitness summary:\n")
```

```
#> Generation 1 fitness summary:
```

```
print(fitness_by_type)
```

```
#> # A tibble: 2 x 4
#>   strategy mean_fitness min_fitness max_fitness
#>   <chr>           <dbl>       <dbl>       <dbl>
#> 1 Dove            0.267         0           0.6
#> 2 Hawk           -0.4          -1.5         1.3
```

```
# Selection: Hawks have lower average fitness when over-represented
cat("\nWith 70% Hawks (above ESS of 40%), Hawks fight each other often.\n")
```

```
#>
#> With 70% Hawks (above ESS of 40%), Hawks fight each other often.
```

```
cat("Doves benefit from facing other Doves more than Hawks benefit from\n")
```

```
#> Doves benefit from facing other Doves more than Hawks benefit from
```

```
cat("fighting other Hawks (since (v-c)/2 < 0 when c > v).\n\n")
```

```
#> fighting other Hawks (since (v-c)/2 < 0 when c > v).
```

```
# Next generation
population_new <- select_parents(population, fitness, n_agents)
population_new <- mutate_population(population_new, mu = 0.01)

cat("After selection and mutation:\n")
```

```
#> After selection and mutation:
```

```
cat(glue("  Hawks: {sum(population_new == 'Hawk')}, ",
         "Doves: {sum(population_new == 'Dove')}"), "\n")
```

```
#>   Hawks: 14, Doves: 6
```

Effect of Mutation Rate on Equilibrium Behavior
Steady-state statistics from last 100 of 300 generations

Mutation rate	Mean Hawk freq	SD of Hawk freq
0.001	0.417	0.045
0.010	0.407	0.052
0.050	0.419	0.040
0.100	0.444	0.041

```
cat(glue("  Hawk frequency: {mean(population_new == 'Hawk')}"), "\n")
```

```
#>   Hawk frequency: 0.7
```

The population moves toward the ESS. Because Hawks are overrepresented (70% vs. the ESS of 40%), they frequently fight each other and incur the costly loss of $(v - c)/2 = -1.5$. Doves, meanwhile, avoid fighting costs entirely. Selection therefore favors Doves, pulling the population toward the equilibrium frequency.

35.4.1. Sensitivity to parameters

```
# Effect of mutation rate on variance of equilibrium frequency
mutation_rates <- c(0.001, 0.01, 0.05, 0.1)

sensitivity_df <- map_dfr(mutation_rates, function(mu) {
  h <- run_hawk_dove_abm(n_agents = 200, n_generations = 300,
                          initial_hawk_freq = 0.5, n_interactions = 10,
                          mutation_rate = mu, v = v, c = c_val, seed = 42)
  # Use last 100 generations as "steady state"
  steady <- tail(h, 100)
  tibble(
    mutation_rate = mu,
    mean_hawk_freq = round(mean(steady$hawk_freq), 3),
    sd_hawk_freq = round(sd(steady$hawk_freq), 3)
  )
})

sensitivity_df |>
  gt() |>
  cols_label(mutation_rate = "Mutation rate",
             mean_hawk_freq = "Mean Hawk freq",
             sd_hawk_freq = "SD of Hawk freq") |>
  tab_header(title = "Effect of Mutation Rate on Equilibrium Behavior",
             subtitle = "Steady-state statistics from last 100 of 300 generations")
```

Higher mutation rates increase the variance around the ESS — agents are more frequently pushed away from the equilibrium — but the mean frequency remains close to $p^* = 0.4$

across all mutation rates. This robustness is a signature of the ESS: selection pressure always pushes back toward equilibrium, regardless of perturbations (von Neumann & Morgenstern, 1944).

35.5. Extensions

- **Spatial structure.** When agents interact only with neighbors on a grid, spatial clusters can form and persist. Cooperation can be sustained in spatial Prisoner's Dilemma even without reciprocity (Chapter 39).
- **Multiple strategies.** The ABM framework extends naturally to games with more than two pure strategies. With three or more strategies, the dynamics can exhibit limit cycles (as in Rock-Paper-Scissors) rather than convergence to a fixed point.
- **Evolutionary stability.** The analytical counterpart to ABM convergence is the concept of an evolutionarily stable strategy. See Chapter 43 for formal definitions and the relationship to Nash equilibrium.
- **Replicator dynamics.** The continuous-time limit of the ABM's selection step yields the replicator equation (Chapter 41). Comparing the ABM to the replicator ODE reveals where finite-population effects matter.
- **Network structure.** When the interaction graph is not well-mixed but has network structure, cooperation dynamics change qualitatively (Chapter 45). For the foundational analysis, see Shoham and Leyton-Brown (2009).

Exercises

1. **Three strategies.** Add a third strategy, **Retaliator**, that plays Dove against Doves and Hawk against Hawks (it matches the opponent's type). This requires each agent to observe its opponent's strategy before acting. Run the ABM and report whether Retaliator can invade a Hawk-Dove population at the ESS.

2. **Population size effects.** Run the Hawk-Dove ABM with population sizes of 20, 100, 500, and 2000 agents for 300 generations each. Plot the standard deviation of the Hawk frequency (over the last 100 generations) against population size. What relationship do you observe, and why?

3. **Stag Hunt dynamics.** Replace the Hawk-Dove payoff matrix with the Stag Hunt: mutual cooperation (Stag, Stag) yields 4 each, mutual defection (Hare, Hare) yields 2 each, and miscoordination yields 0 for the Stag player and 2 for the Hare player. Run the ABM from initial cooperation frequencies of 0.3, 0.5, and 0.8. Does the population always converge to the same equilibrium? Explain in terms of the game's two Nash equilibria.

Solutions appear in Appendix H.

36. Replicating Axelrod's Tournament

A computational replication of Axelrod's 1980 iterated Prisoner's Dilemma tournament in R, exploring why Tit for Tat won and what it teaches about the evolution of cooperation.

37. Replicating Axelrod's Tournament

Learning objectives

- Describe the design of Axelrod's 1980 iterated Prisoner's Dilemma tournament and its key findings.
- Implement a round-robin tournament harness in R with configurable strategies.
- Reproduce the result that Tit for Tat outperforms more complex strategies.
- Visualize tournament outcomes and analyze what makes a strategy successful.

37.1. Motivation

In 1980, political scientist Robert Axelrod invited game theorists to submit strategies for an iterated Prisoner's Dilemma computer tournament. Each strategy would play every other strategy (and itself) in a round-robin, accumulating points over 200 rounds per match. The submissions ranged from simple to elaborate — some used sophisticated pattern recognition, others were just a few lines of code.

The winner was the simplest strategy entered: **Tit for Tat**, submitted by Anatol Rapoport. It cooperates on the first move and then copies whatever the opponent did on the previous move. Axelrod (1981) ran a second tournament after publishing the first results, inviting even more entries with knowledge of the first outcome. Tit for Tat won again.

This result — that a strategy requiring no memory beyond the last round, no exploitation, and no complex reasoning could outperform sophisticated alternatives — became one of the most influential findings in the study of cooperation. In this chapter, we replicate the tournament in R.

37.2. Theory

37.2.1. The iterated Prisoner's Dilemma

The stage game payoffs follow the standard convention:

Table 37.1.: Prisoner's Dilemma payoff structure with $T > R > P > S$ and $2R > T + S$.

	Cooperate	Defect
Cooperate	R, R	S, T
Defect	T, S	P, P

With the canonical values $T = 5$, $R = 3$, $P = 1$, $S = 0$: temptation to defect is high, but mutual cooperation ($R = 3$) beats the average of exploitation and being exploited ($(T + S)/2 = 2.5$).

When the game is repeated, players can condition their current action on the history of play. This opens the door to conditional cooperation: a player might cooperate as long as the opponent cooperates, but punish defection.

37.2.2. Axelrod's four properties of successful strategies

Analyzing the tournament results, Axelrod identified four properties shared by the top-performing strategies:

1. **Nice** — never the first to defect.
2. **Retaliatory** — punish defection promptly.
3. **Forgiving** — return to cooperation after the opponent does.
4. **Clear** — behavior is predictable, enabling mutual cooperation.

Tit for Tat exemplifies all four: it never defects first (nice), defects immediately after the opponent does (retaliatory), cooperates immediately when the opponent returns to cooperation (forgiving), and its logic is transparent (clear).

37.3. Implementation in R

37.3.1. Strategy definitions

We define a library of classic strategies. Each strategy is a function that takes two arguments — the player's own history and the opponent's history — and returns `"C"` or `"D"`.

```r
strategy_tit_for_two_tats <- function(own, opp) {
  n <- length(opp)
  if (n < 2) return("C")
  if (opp[n] == "D" && opp[n - 1] == "D") "D" else "C"
}

strategy_suspicious_tft <- function(own, opp) {
  if (length(opp) == 0) return("D")
  opp[length(opp)]
}

strategy_grudger <- function(own, opp) {
  if (any(opp == "D")) "D" else "C"
}

strategy_pavlov <- function(own, opp) {
  if (length(own) == 0) return("C")
  last_own <- own[length(own)]
  last_opp <- opp[length(opp)]
  if (last_own == last_opp) "C" else "D"
}

strategy_random <- function(own, opp) {
  sample(c("C", "D"), 1)
}
```

```
strategies <- list(
  "Tit for Tat"        = strategy_tit_for_tat,
  "Always Cooperate"   = strategy_always_cooperate,
  "Always Defect"      = strategy_always_defect,
  "Tit for Two Tats"   = strategy_tit_for_two_tats,
  "Suspicious TFT"     = strategy_suspicious_tft,
  "Grudger"            = strategy_grudger,
  "Pavlov"             = strategy_pavlov,
  "Random"             = strategy_random
)
```

37.3.2. Round-robin tournament

```
run_tournament <- function(strategies, rounds = 200) {
  names_s <- names(strategies)
  n <- length(strategies)
  results <- matrix(0, nrow = n, ncol = n,
                    dimnames = list(names_s, names_s))
  for (i in seq_len(n)) {
    for (j in seq(i, n)) {
      scores <- run_match(strategies[[i]], strategies[[j]], rounds)
      results[i, j] <- results[i, j] + scores[1]
      results[j, i] <- results[j, i] + scores[2]
    }
  }
  results
}

set.seed(42)
results <- run_tournament(strategies, rounds = 200)

totals <- rowSums(results)
ranking <- sort(totals, decreasing = TRUE)

cat("Tournament Results (total scores):\n\n")
```

```
#> Tournament Results (total scores):
```

```
for (i in seq_along(ranking)) {
  cat(sprintf("  %d. %-20s %5d\n", i, names(ranking)[i], ranking[i]))
}
```

```
#>   1. Tit for Two Tats      4761
#>   2. Tit for Tat           4725
#>   3. Grudger               4585
#>   4. Pavlov                4538
#>   5. Always Cooperate      4518
#>   6. Random                3900
```

```
#>    7. Always Defect          3416
#>    8. Suspicious TFT         3372
```

37.3.3. Visualizing the results

```
ranking_df <- tibble(
  strategy = factor(names(ranking), levels = rev(names(ranking))),
  score = as.integer(ranking)
)

nice_strategies <- c("Tit for Tat", "Always Cooperate", "Tit for Two Tats",
                     "Grudger", "Pavlov")
ranking_df$nice <- names(ranking) %in% nice_strategies

p_ranking <- ggplot(ranking_df, aes(x = score, y = strategy, fill = nice)) +
  geom_col(width = 0.7) +
  scale_fill_manual(values = c("TRUE" = okabe_ito[3], "FALSE" = okabe_ito[6]),
                    labels = c("TRUE" = "Nice", "FALSE" = "Not nice"),
                    name = "Strategy type") +
  theme_publication() +
  labs(title = "Axelrod Tournament: Strategy Rankings",
       x = "Total score", y = NULL)

p_ranking
save_pub_fig(p_ranking, "axelrod-ranking", width = 7, height = 5)
```

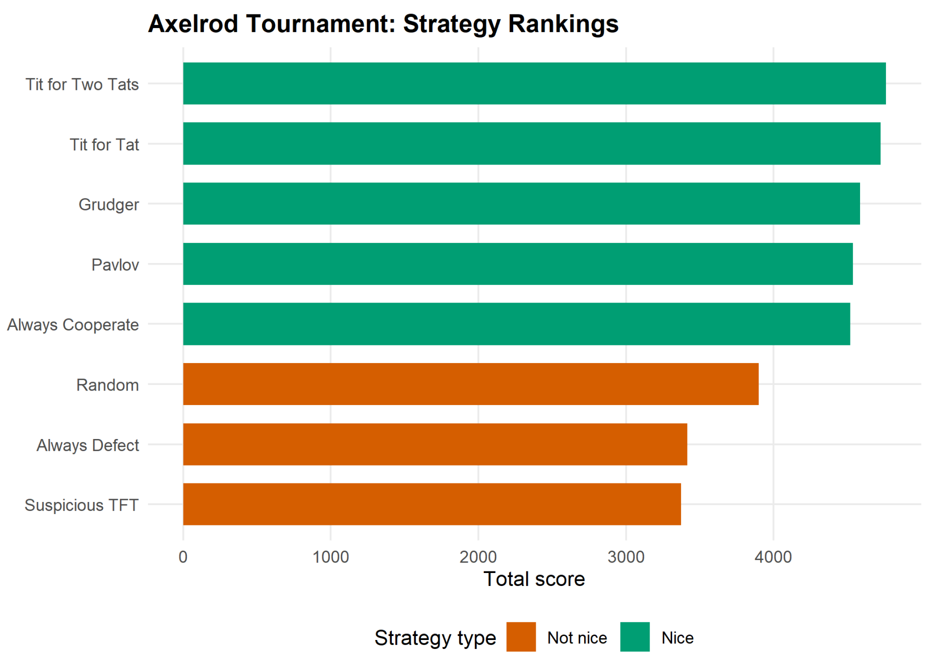

Figure 37.1.: Tournament ranking by total score. Tit for Tat wins the round-robin tournament, consistent with Axelrod's original finding. Nice strategies (those that never defect first) cluster at the top.

37.3.4. Head-to-head heatmap

```
heatmap_df <- as.data.frame(as.table(results))
names(heatmap_df) <- c("Player", "Opponent", "Score")

p_heat <- ggplot(heatmap_df, aes(x = Opponent, y = Player, fill = Score)) +
  geom_tile(colour = "white", linewidth = 0.5) +
  geom_text(aes(label = Score), size = 2.8) +
  scale_fill_gradient(low = okabe_ito[2], high = okabe_ito[1]) +
  theme_publication(base_size = 10) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  labs(title = "Head-to-Head Scores",
       x = "Opponent", y = "Player")

p_heat
save_pub_fig(p_heat, "axelrod-heatmap", width = 7, height = 6)
```

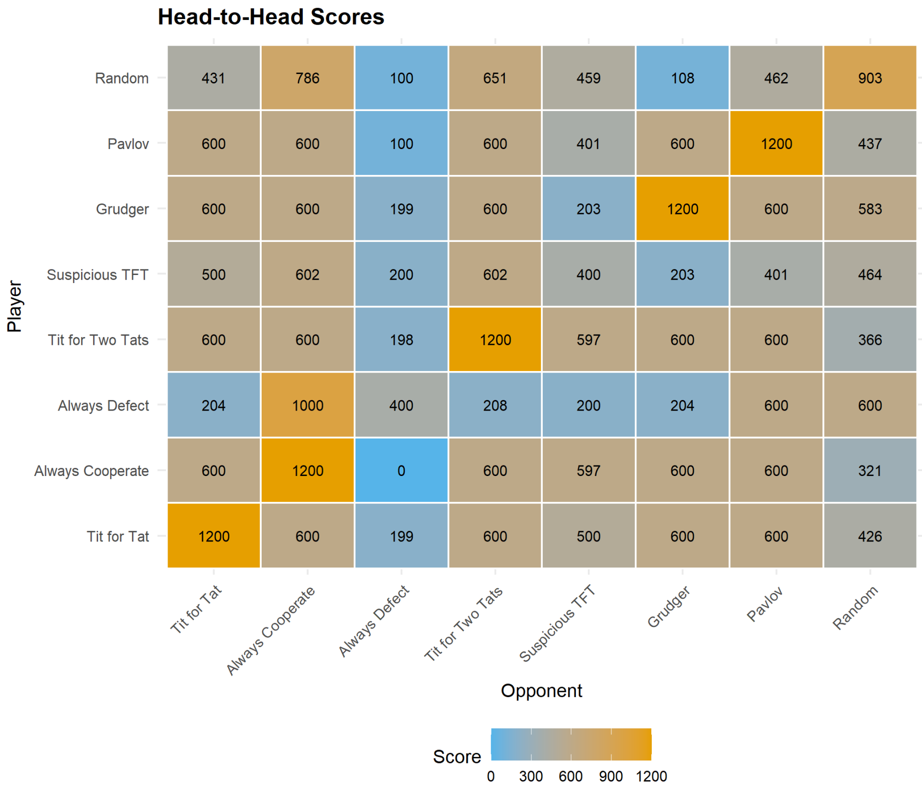

Figure 37.2.: Head-to-head scores in the round-robin tournament. Rows are the focal player; columns are the opponent. Always Defect scores highest against Always Cooperate but poorly against retaliatory strategies.

37.4. Worked example

Let us trace a single match between Tit for Tat and Suspicious TFT (which defects first, then copies):

```
set.seed(42)
h1 <- character(0)
h2 <- character(0)

for (r in 1:10) {
  a1 <- strategy_tit_for_tat(h1, h2)
  a2 <- strategy_suspicious_tft(h2, h1)
  h1 <- c(h1, a1)
  h2 <- c(h2, a2)
}

trace_df <- tibble(
  Round = 1:10,
  `Tit for Tat` = h1,
  `Suspicious TFT` = h2
)

cat("First 10 rounds: Tit for Tat vs. Suspicious TFT\n\n")
```

```
#> First 10 rounds: Tit for Tat vs. Suspicious TFT
```

```
print(trace_df, n = 10)
```

```
#> # A tibble: 10 x 3
#>    Round `Tit for Tat` `Suspicious TFT`
#>    <int> <chr>         <chr>
#>  1     1 C             D
#>  2     2 D             C
#>  3     3 C             D
#>  4     4 D             C
#>  5     5 C             D
#>  6     6 D             C
#>  7     7 C             D
#>  8     8 D             C
#>  9     9 C             D
#> 10    10 D             C
```

The pattern reveals alternating retaliation: Suspicious TFT defects in round 1, Tit for Tat retaliates in round 2, and the two lock into a cycle of mutual recrimination. This is a well-known weakness of Tit for Tat — it can get trapped in defection cycles against strategies that defect first. Tit for Two Tats avoids this by requiring two consecutive defections before retaliating, at the cost of being more exploitable.

37.5. Extensions

- **Evolutionary dynamics**: What happens when successful strategies reproduce and unsuccessful ones die out? This leads to replicator dynamics on strategy populations (Chapter 41) and the concept of evolutionarily stable strategies (Chapter 43).

- **Spatial structure**: When players interact only with neighbors on a lattice, cooperation can survive even without reciprocity (Chapter 39).
- **Noise**: In noisy environments where actions are sometimes misperceived, Tit for Tat's strict reciprocity becomes a liability. Generous Tit for Tat (cooperate with some probability after opponent's defection) outperforms in noisy settings.
- For the original analysis, see Axelrod (1981). For a modern computational treatment with hundreds of strategies, see the Python `axelrod` library (accessible via `reticulate`; see Chapter 29 for the general pattern).

Exercises

1. **Add a strategy.** Implement a "Gradual" strategy that retaliates with an increasing number of defections after each opponent defection (1 defection after the first, 2 after the second, etc.), then returns to cooperation. Add it to the tournament and report the new rankings.
2. **Vary the number of rounds.** Run the tournament with 50, 200, and 1000 rounds. How do the rankings change? Why might Always Defect do better in shorter tournaments?
3. **Cooperation rate.** Modify `run_match()` to return the cooperation rate (fraction of rounds where both players cooperated) in addition to scores. Which pair of strategies achieves the highest mutual cooperation rate? Which achieves the lowest?

Solutions appear in Appendix H.

38. The Spatial Prisoner's Dilemma

A computational exploration of Nowak and May's spatial Prisoner's Dilemma, showing how local interaction on a lattice allows cooperation to survive through cluster formation — even without memory, reciprocity, or reputation.

39. Spatial Prisoner's Dilemma

Learning objectives

- Explain why spatial structure can sustain cooperation in the Prisoner's Dilemma without reciprocity or punishment.
- Implement a lattice-based spatial PD with Moore neighborhood interaction and imitation dynamics in R.
- Visualize the emergence and persistence of cooperator clusters on a grid.
- Analyze how the fraction of cooperators evolves over time under different payoff parameters.

39.1. Motivation

In Chapter 37, cooperation emerged because players could remember past interactions and condition their behavior on history. But many biological and social systems lack this luxury. Bacteria, plants, and even commuters on a highway interact with nearby neighbors without tracking individual histories. Can cooperation survive when there is no memory, no reputation, and no punishment — only spatial structure?

In 1992, Martin Nowak and Robert May (1992) showed that the answer is yes. They placed players on a two-dimensional grid where each cell plays a one-shot Prisoner's Dilemma with its immediate neighbors. After each round, every cell adopts the strategy of whichever neighbor (including itself) scored highest. The result was striking: cooperators formed compact clusters that could resist invasion by defectors, producing intricate, dynamic spatial patterns.

This chapter replicates their model in R. We will build a lattice simulation, watch cooperation clusters emerge, and measure how population-level cooperation rates evolve over time.

39.2. Theory

39.2.1. From well-mixed to spatial populations

Standard evolutionary game theory assumes a **well-mixed population**: every individual is equally likely to interact with every other. Under this assumption, defection dominates in a one-shot Prisoner's Dilemma because a defector always earns more than a cooperator against any given opponent.

Spatial structure breaks this assumption. When interaction is **local** — each player meets only its geographic neighbors — cooperators can form clusters. Within a cluster, cooperators interact mainly with other cooperators, earning the mutual cooperation payoff R many times. A defector on the boundary of such a cluster exploits its cooperator neighbors but has fewer cooperator neighbors than cooperators in the cluster interior do.

39.2.2. The Nowak–May model

The model works as follows:

1. **Grid**: An $N \times N$ lattice with periodic boundary conditions (a torus, so edges wrap around).
2. **Strategies**: Each cell is either a cooperator (C) or a defector (D).
3. **Interaction**: Each cell plays a one-shot PD with each of its eight Moore neighbors and itself (nine games total). The cell's score is the sum of payoffs from all nine games.
4. **Update**: Synchronously, every cell adopts the strategy of the highest-scoring cell in its Moore neighborhood (including itself). Ties are broken in favor of the current occupant.

With canonical PD payoffs ($T > R > P > S$), the dynamics depend critically on the temptation-to-defect parameter $b = T/R$. For moderate values of b, cooperators and defectors coexist in dynamic spatial patterns. For very high b, defection takes over; for low b, cooperation dominates.

39.2.3. Why clusters protect cooperators

Consider a 3×3 block of cooperators surrounded by defectors. The cooperator at the center plays nine cooperators (including itself), scoring $9R$. A defector adjacent to this block has at most three cooperator neighbors, scoring at most $3T + 6P$. When $9R > 3T + 6P$ — that is, when $b < 3(R - P/R) + P$ approximately — the interior cooperator outscores the boundary defector, and the cluster holds. This is the geometric logic behind spatial cooperation.

39.3. Implementation in R

39.3.1. Grid initialization and neighbor scoring

```
initialize_grid <- function(n, prob_coop = 0.5) {
  matrix(sample(c("C", "D"), n * n, replace = TRUE,
                prob = c(prob_coop, 1 - prob_coop)),
         nrow = n, ncol = n)
}

get_moore_neighbors <- function(grid, i, j) {
  n <- nrow(grid)
  rows <- ((i - 2):(i)) %% n + 1
  cols <- ((j - 2):(j)) %% n + 1
  grid[rows, cols]
}

score_cell <- function(grid, i, j) {
  neighbors <- get_moore_neighbors(grid, i, j)
  focal <- grid[i, j]
  total <- 0
  for (s in as.vector(neighbors)) {
    total <- total + play_round(focal, s)[1]
  }
```

```
  total
}

compute_scores <- function(grid) {
  n <- nrow(grid)
  scores <- matrix(0, nrow = n, ncol = n)
  for (i in seq_len(n)) {
    for (j in seq_len(n)) {
      scores[i, j] <- score_cell(grid, i, j)
    }
  }
  scores
}
```

39.3.2. Update rule: imitate the best neighbor

```
update_grid <- function(grid, scores) {
  n <- nrow(grid)
  new_grid <- grid
  for (i in seq_len(n)) {
    for (j in seq_len(n)) {
      rows <- ((i - 2):(i)) %% n + 1
      cols <- ((j - 2):(j)) %% n + 1
      neighborhood_scores <- scores[rows, cols]
      best <- which(neighborhood_scores == max(neighborhood_scores),
                    arr.ind = TRUE)
      if (nrow(best) == 1) {
        bi <- rows[best[1, 1]]
        bj <- cols[best[1, 2]]
        new_grid[i, j] <- grid[bi, bj]
      }
      # Tie: keep current strategy (new_grid[i,j] already equals grid[i,j])
    }
  }
  new_grid
}
```

39.3.3. Running the simulation

```
run_spatial_pd <- function(n, generations, prob_coop = 0.5, seed = 42) {
  set.seed(seed)
  grid <- initialize_grid(n, prob_coop)

  history <- vector("list", generations + 1)
  history[[1]] <- grid
  coop_frac <- numeric(generations + 1)
  coop_frac[1] <- mean(grid == "C")
```

```
  for (g in seq_len(generations)) {
    scores <- compute_scores(grid)
    grid <- update_grid(grid, scores)
    history[[g + 1]] <- grid
    coop_frac[g + 1] <- mean(grid == "C")
  }

  list(history = history, coop_frac = coop_frac)
}
```

39.3.4. Visualizing a grid snapshot

```
grid_to_df <- function(grid, gen) {
  n <- nrow(grid)
  expand.grid(row = seq_len(n), col = seq_len(n)) |>
    mutate(strategy = as.vector(grid),
           generation = gen)
}
```

39.4. Worked example

We simulate a 30×30 grid with 50% initial cooperators for 100 generations, using the canonical PD payoffs ($T = 5$, $R = 3$, $P = 1$, $S = 0$).

```
result <- run_spatial_pd(n = 30, generations = 100, prob_coop = 0.5, seed = 42)
```

```
snapshot_gens <- c(0, 5, 20, 100)
snapshot_df <- map_dfr(snapshot_gens, function(g) {
  grid_to_df(result$history[[g + 1]], g)
})
snapshot_df$generation <- factor(
  snapshot_df$generation,
  levels = snapshot_gens,
  labels = paste("Generation", snapshot_gens)
)

p_snapshots <- ggplot(snapshot_df,
                      aes(x = col, y = row, fill = strategy)) +
  geom_tile() +
  facet_wrap(~generation, ncol = 2) +
  scale_fill_manual(values = c("C" = okabe_ito[1], "D" = okabe_ito[5]),
                    labels = c("C" = "Cooperate", "D" = "Defect"),
                    name = "Strategy") +
  coord_equal() +
  theme_publication() +
  theme(axis.text = element_blank(),
        axis.ticks = element_blank(),
```